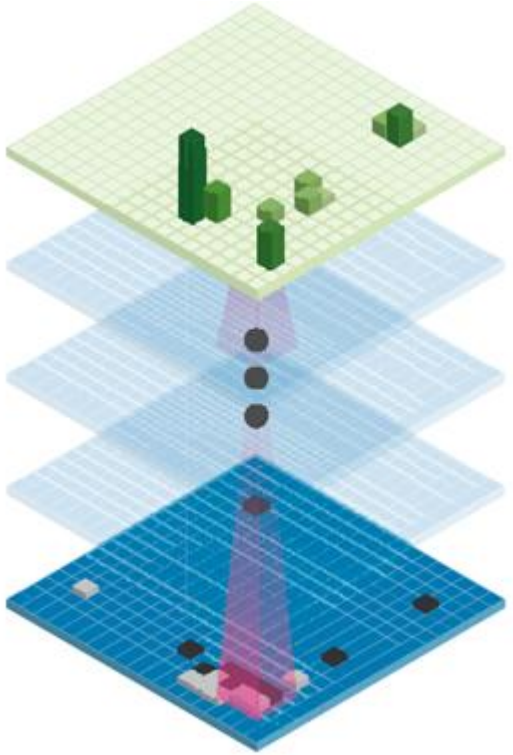


第4章 串和数组

主要内容



4.1 串的基本概念、抽象描述和分类

4.2 串的模式匹配

4.3 数组的概念、特性、遍历

4.4 特殊矩阵的压缩存储

4.5 总结

4.1 串的基本概念、抽象描述和分类

——4.1.1 串的基本概念——

字符串也叫串，是由字符组成的有限序列，是一种常用的非数值数据。串的逻辑结构是线性表，串是一种特殊的线性表，其每个数据元素都是一个字符。但是串的操作特点与线性表不同，主要是对子串进行操作，通常采用顺序存储结构存储。

字符串可以表示为str="a0a1…ai…an-1"，其中str为串名，也叫串变量； i为字符ai在串中的位序号； 双引号中的字符序列称为串值； n为串的长度； 当n=0时字符串不包含任何字符，为空串； 当字符串由一个或多个空白字符组成时为空白串。

——4.1.1 串的基本概念——

字符串中任意个连续字符组成的子序列称为字符串的子串，此字符串为该子串的主串。子串在主串中的位置是指子串在主串中第一次出现时的第一个字符在主串中的位置。空串是任意串的子串，每个字符串都是其自身的子串，除自身外，主串的其他子串称为主串的真子串。

串的比较规则和字符的比较规则有关，字符的比较规则由所属的字符集的编码决定。两个串相等是指串长度相同并且各对应位置上的字符也相同。两个串的大小由对应位置上的首个不同字符的大小决定，字符比较次序是从头开始依次向后。当两个串的长度不等而对应位置上的字符都相同时较长的串定义为较大。

4.1.2 串的抽象数据类型描述

```python
from abc import ABCMeta, abstractmethod, abstractproperty

class IString(metaclass=ABCMeta):
    @abstractmethod
    def clear(self):
        '''将字符串置成空串'''
        pass

    @abstractmethod
    def isEmpty(self):
        '''判断是否为空串'''
        pass

    @abstractmethod
    def length(self):
        '''返回串的长度'''
        pass

    @abstractmethod
    def charAt(self, i):
        '''读取并返回串中的第i个数据元素'''
        pass

    @abstractmethod
    def subString(self, begin, end):
        '''返回位序号从begin到end-1的子串'''
        pass

    @abstractmethod
    def insert(self, i, str):
        '''在第i个字符之前插入子串str'''
        pass

    @abstractmethod
    def delete(self, begin, end):
        '''删除位序号从begin到end-1的子串'''
        pass

    @abstractmethod
    def concat(self, str):
        '''将str连接到字符串的后面'''
        pass

    @abstractmethod
    def compareTo(self, str):
        '''比较str和当前字符串的大小'''
        pass

    @abstractmethod
    def indexOf(self, str, begin):
        '''从位序号为begin的字符开始搜索与str相等的子串'''
        pass
```

字符串是数据元素类型为字符的线性表，其抽象数据类型描述与线性表相似，又根据串在实际问题中的应用抽象出串的基本操作，可得串的抽象数据类型Python语言描述如左边所示。

4.1.2 串的抽象数据类型描述

```python
from abc import ABCMeta, abstractmethod, abstractproperty

class IString(metaclass=ABCMeta):
    @abstractmethod
    def clear(self):
        '''将字符串置成空串'''
        pass

    @abstractmethod
    def isEmpty(self):
        '''判断是否为空串'''
        pass

    @abstractmethod
    def length(self):
        '''返回串的长度'''
        pass

    @abstractmethod
    def charAt(self, i):
        '''读取并返回串中的第i个数据元素'''
        pass

    @abstractmethod
    def subString(self, begin, end):
        '''返回位序号从begin到end-1的子串'''
        pass

    @abstractmethod
    def insert(self, i, str):
        '''在第i个字符之前插入子串str'''
        pass

    @abstractmethod
    def delete(self, begin, end):
        '''删除位序号从begin到end-1的子串'''
        pass

    @abstractmethod
    def concat(self, str):
        '''将str连接到字符串的后面'''
        pass

    @abstractmethod
    def compareTo(self, str):
        '''比较str和当前字符串的大小'''
        pass

    @abstractmethod
    def indexOf(self, str, begin):
        '''从位序号为begin的字符开始搜索与str相等的子串'''
        pass
```

字符串的抽象数据类型Python抽象类包含了串的主要基本操作，如果要使用这个接口，还需要具体的类来实现。串的Python抽象类的实现方法主要有以下两种：

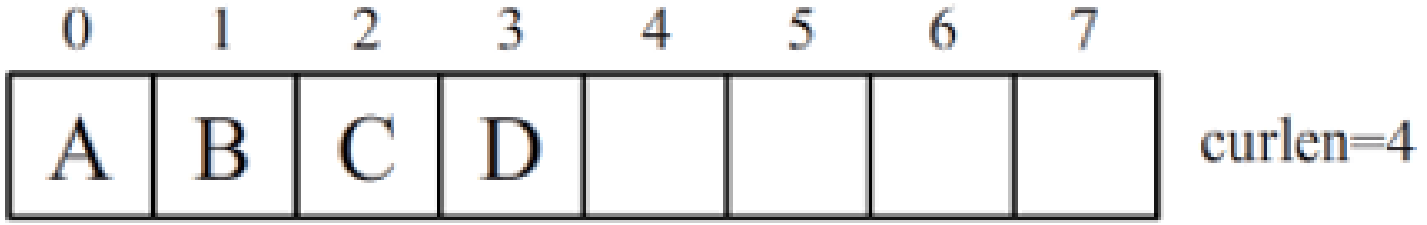
(1) 基于顺序存储的实现，为顺序串；

(2) 基于链式存储的实现，为链串。

——4.1.3 顺序串——

1. 顺序串的描述：

顺序串与顺序表的逻辑结构相同，存储结构类似，均可用数组来存储数据元素。但串具有独特的不同于线性表的操作，属于特殊类型的线性表。下图所示为顺序串。



——4.1.3 顺序串——

实现IString抽象类的顺序串类的Python语言描述如下：

```python
class SqString(IString):
    def __init__(self,obj=None):
        if obj is None: # 构造空串
            self.strValue = [] # 字符数组存放串值
            self.curLen = 0 # 当前串的长度
        elif isinstance(obj,str): # 以字符串构造串
            self.curLen = len(obj)
            self.strValue = [None] * self.curLen
            for i in range(self.curLen):
                self.strValue[i] = obj[i]
        elif isinstance(obj,list): # 以字符列表构造串
            self.curLen = len(obj)
            self.strValue = [None] * self.curLen
            for i in range(self.curLen):
                self.strValue[i] = obj[i]

    def clear(self):
        '''将字符串置成空串'''
        self.curLen = 0

    def isEmpty(self):
        '''判断是否为空串'''
        return self.curLen == 0

    def length(self):
        '''返回串的长度'''
        return self.curLen

    def charAt(self,i):
        '''读取并返回串中的第i个数据元素'''
        if i<0 or i>=self.curLen:
            raise IndexError("String index out of range")
        return self.strValue[i]

```

```python
    def allocate(self,newCapacity):
        '''将串的长度扩充为newCapacity'''
        tmp = self.strValue
        self.strValue = [None] * newCapacity
        for i in range(self.curLen):
            self.strValue[i] = tmp[i]

    def subString(self,begin,end):
        '''返回位序号从begin到end-1的子串'''
        pass

    def insert(self,i,str):
        '''在第i个字符之前插入子串str'''
        pass

    def delete(self,begin,end):
        '''删除位序号从begin到end-1的子串'''
        pass

    def concat(self,str):
        '''将str连接到字符串的后面'''
        pass

    def compareTo(self,str):
        '''比较str和当前字符串的大小'''
        pass

    def indexOf(self,str,begin):
        '''从位序号为begin的字符开始搜索与str相等的子串'''
        pass

    def display(self):
        '''打印字符串'''
        for i in range(self.curLen):
            print(self.strValue[i],end='')
```

——4.1.3 顺序串——

2. 顺序串基本操作的实现

顺序串有许多基本操作，例如，求子串操作、插入操作、删除操作、连接操作、比较操作。

下面，我们对这几个操作逐个使用Python实现。

——4.1.3 顺序串——

1) 求子串操作

```
1  def subString(self,begin,end):
2      '''返回位序号从begin到end-1的子串'''
3      if begin<0 or begin>=end or end>self.curLen:
4          raise IndexError("参数不合法")
5      tmp = [None] * (end-begin)
6      for i in range(begin,end):
7          tmp[i-begin] = self.strValue[i] # 复制子串
8      return SqString(tmp)
```

求子串操作subString(begin,end)是返回长度为n的字符串中位序号从begin到end-1的字符序列，其中0≤begin≤n-1，begin<end≤n。其主要步骤如下：

(1) 检查参数begin和end是否满足0≤begin≤n-1和begin<end≤n，若不满足，抛出异常。

(2) 返回位序号为begin到end-1的字符序列。

代码如左图所示

——4.1.3 顺序串——

2) 插入操作

```
def insert(self,i,str):
    '''在第i个字符之前插入子串str'''
    if i<0 or i>self.curLen:
        raise IndexError("插入位置不合法")
    length = str.length()
    newCapacity = self.curLen + length
    self.allocate(newCapacity)
    for j in range(self.curLen-1,i-1,-1):
        self.strValue[j+length] = self.strValue[j]
    for j in range(i,i+length):
        # print(j-i,str.charAt(j-i))
        self.strValue[j] = str.charAt(j-i)
    self.curLen = newCapacity
```

插入操作insert(i,str)是在长度为n的字符串的第i个元素之前插入串str，其中0≤i≤n。其主要步骤如下：

(1) 判断参数i是否满足0≤i≤n，若不满足，则抛出异常。

(2) 重新分配存储空间为n+m，m为插入的字符串str的长度。

(3) 将第i个及之后的数据元素向后移动m个存储单元。

(4) 将str插入到字符串从i开始的位置。

——4.1.3 顺序串——

3) 删除操作

```
1   def delete(self,begin,end):
2       '''删除位序号从begin到end-1的子串'''
3       if begin<0 or begin>=end or end>self.curLen:
4           raise IndexError("参数不合法")
5       for i in range(begin,end):
6           self.strValue[i] = self.strValue[i+end-begin]
7       self.curLen = self.curLen - end + begin
```

删除操作delete(begin,end)是将长度为n的字符串的位序号为begin到end-1的元素删除，其中参数begin和end满足0≤begin≤curLen-1和begin<end≤curLen。其主要步骤如下：

(1) 判断参数begin和end是否满足0≤begin≤curLen-1和begin< end ≤curLen，若不满足，则抛出异常。

(2) 将字符串位序号为end的数据元素及其之后的数据元素向前移动到位序号为begin的位置。

——4.1.3 顺序串——

4) 连接操作

5) 比较操作

```
def compareTo(self,str):
    '''比较str和当前字符串的大小'''
    n = self.curLen if self.curLen<str.length() else str.length()
    for i in range(n):
        if self.strValue[i]>str.charAt(i):
            return 1
        if self.strValue[i]<str.charAt(i):
            return -1
    if self.curLen > str.length():
        return 1
    elif self.curLen < str.length():
        return -1
    return 0
```

4) concat(str)是将串str插入到字符串的尾部，此时调用insert(curLen,str)即可实现。

5) 比较操作compareTo(str)是将字符串与串str按照字典序进行比较。若当前字符串较大，返回1； 若相等，返回0。若当前字符串较小，返回-1。其主要步骤如下：

(1) 确定需要比较的字符的个数n为两个字符串长度的较小值。

(2) 从下标0至n-1依次进行比较。

——4.1.3 顺序串——

【例4.1】编写一个程序，完成构造串、判断串是否为空、返回串的长度、求子串等操作。

示例答案：

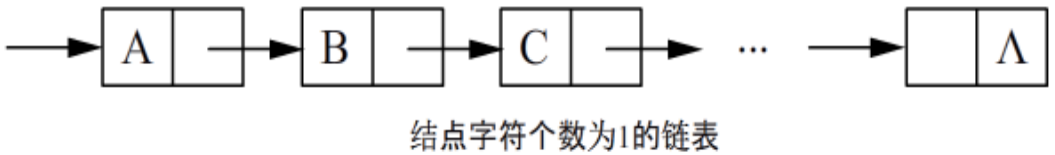
```
s1 = SqString(['1', '2', '3', '4', '5', '6'])
s2 = SqString(['1', '2', '3', '4', '5', '6'])
s1.insert(1,s2)
s1.display()
print()
s1.delete(1,6)
s1.display()
print()
s1.concat(s2)
s1.display()
print()
s3 = s1.subString(1,6)
s3.display()
print()
print(s1.compareTo(s3))
```

——4.1.4 链串——

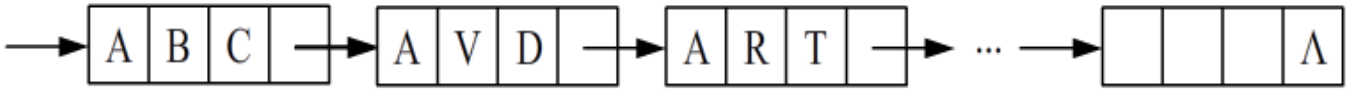
链串的描述：

链串采用链式存储结构，和线性表的链式存储结构类似，可以采用单链表存储串值。链串由一系列大小相同的结点组成，每个结点用数据域存放字符，指针域存放指向下一个结点的指针。

与线性表不同的是每个结点的数据域可以是一个字符或者多个字符。若每个结点的数据域为一个字符，这种链表称为单字符链表； 若每个结点的数据域为多个字符，则称为块链表。在块链表中每个结点的数据域不一定被字符占满，可通过添加空字符或者其他非串值字符来简化操作。如图所示为两种不同类型的链串。



结点字符个数为1的链表



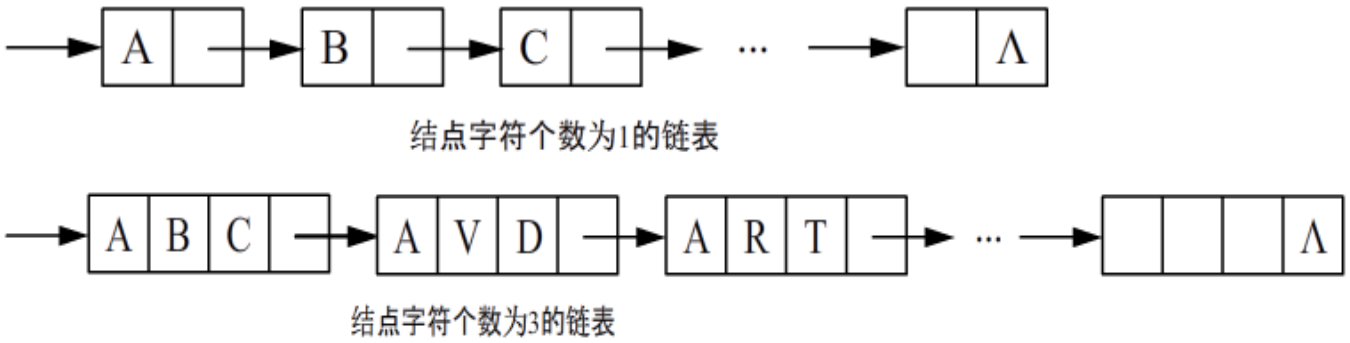
结点字符个数为3的链表

——4.1.4 链串——

链串的优缺点：

在串的链式存储结构中，单字符链表的插入、删除操作较为简单，但存储效率低。块链表虽然存储效率较高但插入、删除操作需要移动字符，较为复杂。此外，与顺序串相比，链串需要从头部开始遍历才能访问某个位置的元素。

所以用户在应用中需要根据实际情况选择合适的存储结构。



4.2 串的模式匹配

4.2 串的模式匹配

串的模式匹配也叫查找定位，指的是在当前串中寻找模式串的过程，主要的模式匹配算法有Brute Force算法和KMP算法。

```
def BF(self,str,begin):
    if str.length()<=self.curLen and str is not None and self.curLen>0:
        i = begin
        length = str.length()
        while(i<=self.curLen-length):
            for j in range(length):
                if str.charAt(j)!=self.strValue[j+i]:
                    i += 1
                    break
                elif j==length-1:
                    return i
    return -1
```

```
def KMP(self,p,begin):
    next = SqString.next(p) # 计算next值
    i = begin # # i为主串的字符指针
    j = 0
    while i < self.curLen and j < p.length():
        if j==-1 or self.strValue[i]==p.charAt(j):
            # 比较的字符相等或者比较主串的下一个字符
            i += 1
            j += 1
        else:
            j = next[j]
    if j == p.length():
        return i-j # 匹配
    else:
        return -1
```

——4.2.1 Brute Force 算法——

```
def BF(self,str,begin):
    if str.length()<=self.curLen and str is not None and self.curLen>0:
        i = begin
        length = str.length()
        while(i<=self.curLen-length):
            for j in range(length):
                if str.charAt(j)!=self.strValue[j+i]:
                    i += 1
                    break
                elif j==length-1:
                    return i
    return -1
```

Brute Force算法从主串的第一个字符开始和模式串的第一个字符进行比较，若相等，则继续比较后续字符； 否则从主串的第二个字符开始重新和模式串进行比较。依此类推，直到模式串的每个字符依次与主串的字符相等，匹配成功。

——4.2.1 Brute Force 算法——

Brute Force算法的实现简单，但效率非常低。m为模式串的长度，n为主串的长度。

（1） 最好情况： 第一次匹配即成功，比较次数为模式串的长度m，时间复杂度为O(m)。

（2） 最坏情况： 每次匹配比较至模式串的最后一个字符，并且比较了目标串中所有长度为m的子串，此时的时间复杂度为O(m×n)。

缺点：每次匹配没有利用前一次匹配的比较结果，使算法中存在较多的重复比较，降低了算法的效率；如果利用部分字符匹配的结果，可将算法的效率提高。因此提出了KMP算法，在下一节进行介绍。

```
1   def BF(self,str,begin):
2       if str.length()<=self.curLen and str is not None and self.curLen>0:
3           i = begin
4           length = str.length()
5           while(i<=self.curLen-length):
6               for j in range(length):
7                   if str.charAt(j)!=self.strValue[j+i]:
8                       i += 1
9                       break
10                  elif j==length-1:
11                      return i
12      return -1
```

——4.2 串的模式匹配——

1. 算法分析

设主串为s="aba bcabdabcabca"、模式串为p="abcabc"，指针i、j分别指示主串和模式串所比较字符的位序号。当某次匹配不成功时有si≠pj，并且si-jsi-j+1…si-1=p0p1…pj-1。此时需要寻找前缀子串p0p1…pk-1=pj-kpj-k+1…pj-1，其中0<k<j，这时候即满足si-ksi-k+1…si-1=p0p1…pk-1，下一次匹配可直接比较si和pk。此外，为了减少比较次数，k应取最大值，即p0p1…pk-1应为满足此性质的最长前缀子串。若k不存在，下一次匹配则直接比较si和p0。

——4.2.2 KMP 算法——

2. K值的计算

通过前面的分析已知，每次模式串开始比较的位置（即k的值）仅与模式串本身有关。一般用next[j]来表示pj对应的k值。

初始时可定义next[0]=-1，next[1]=0。

设next[j]=k，则p0p1…pk-1=pj-kpj-k+1…pj-1，k为满足等式的最大值。计算next[j+1]的值。

(1) 若pk=pj，则存在p0p1…pk-1pk=pj-kpj-k+1…pj-1pj，此时next[j+1]=k+1。

(2) 若pk≠pj，可以把计算next[j]的值的问题看成新的模式匹配过程，主串为p，模式串为p的前缀子串。

出现不匹配，应将模式串的比较位置变为k′=next[k]，若pj=p_(k^')，则next[j+1]=k′+1=next[k]+1，否则继续执行步骤(2)，直到pj=pk，或者当k=0并且pj≠pk时next[j+1]=0。

——4.2.2 KMP 算法——

2. K值的计算

代码实现：

```
1   def next(p):
2       next = [0] * p.length() # next数组
3       k = 0 # 模式串指针
4       j = 1 # 主串指针
5       next[0] = -1
6       next[1] = 0
7       while j<p.length()-1:
8           if p.charAt(j)==p.charAt(k):
9               next[j+1] = k+1
10              k += 1
11              j += 1
12          elif k==0:
13              next[j+1] = 0
14              j += 1
15          else:
16              k = next[k]
17      return next
```

——4.2.2 KMP 算法——

3. KMP算法步骤

```
1   def KMP(self,p,begin):
2       next = SqString.next(p) # 计算next值
3       i = begin # # i为主串的字符指针
4       j = 0
5       while i < self.curLen and j < p.length():
6           if j==-1 or self.strValue[i]==p.charAt(j):
7               # 比较的字符相等或者比较主串的下一个字符
8               i += 1
9               j += 1
10          else:
11              j = next[j]
12      if j == p.length():
13          return i-j # 匹配
14      else:
15          return -1
```

设主串的长度为n、模式串的长度为m，求next[]的时间复杂度为O(m)。在KMP中，因主串的下标不需要回退，比较次数最多为n-m+1，所以KMP算法的时间复杂度为O(m+n)。

KMP算法的主要步骤如下。

(1) 计算模式串的next[]函数值。

(2) i为主串的比较字符位序号，j为模式串的比较字符位序号。当字符相等时，i、j分别加1后继续比较； 否则i的值不变，j=next[j]，继续比较。

(3) 重复步骤(2)，直到j等于模式串的长度时匹配成功，否则匹配失败。

——4.2.2 KMP 算法——

3. KMP算法步骤

小测试：请计算 str = “abcababc”的next[j] 的值

参考答案：

当j=0时，next[0]=-1;
当j=1时，next[1]=0;
当j=2时，next[2]=0;
当j=3时，next[3]=0;
当j=4时，next[4]=1;
当j=5时，next[5]=2;
当j=6时，next[6]=1;
当j=7时，next[7]=2。

4.3 数组的概念、特性、遍历

——4.3.1 数组的基本概念——

数组是n个具有相同数据类型的数据元素构成的集合，数组元素按某种次序存储在地址连续的存储单元中,是顺序存储的随机存储结构。

数组元素在数组中的位置称为数组元素的下标，用户通过下标可以访问相应的数组元素。数组下标的个数是数组的维数，具有一个下标的数组叫一维数组，具有两个下标的数组叫二维数组。一维数组的逻辑结构是线性表，多维数组是线性表的扩展。二维数组可以看成数组元素是一维数组的数组。下图所示为二维数组的矩阵表示。

——4.3.1 数组的基本概念——

二维数组中的每个数据元素ai,j都受到两个关系的约束，即行关系和列关系。ai.,j+1是ai,j在行关系中的后继元素； ai+1,j是ai,j在列关系中的后继元素。

因为二维数组可以看成数组元素是一维数组的数组，所以二维数组也可看成线性表，即A=(a0,a1,...,an-1)，其中每个数据元素ai是一个列向量的线性表，即ai=(a0i,a1i,...,am-1i);或者表述为A=(a0,a1,...,am-1)，其中每个数据元素ai是一个行向量的线性表，即ai=(a0i,a1i,...,an-1i)。其中，每个元素同时属于两个线性表，第i行的线性表和第j列的线性表，具体可以分析如下：

（1） a0,0是起点，没有前驱元素； am-1,n-1是终点，没有后继元素。

（2） 边界元素ai,0和a0,j（1≤j<n,1≤i<m）只有一个前驱元素； ai,n-1和am-1,j（0≤j<n-1,1≤i<m-1）只有一个后继元素。

（3） ai,j（1≤j<n-1,1≤i<m-1）有两个前驱元素和两个后继元素。

——4.3.2 数组的特性——

数组元素被存放在一组地址连续的存储单元里，并且每个数据元素的大小相同，故只要已知首地址和每个数据元素占用的内存单元大小即可求出数组中任意数据元素的存储地址。

对于**一维数组**A[n]，数据元素的存储地址为Loc(i)=Loc(0)+i×L(0≤i<n)，其中Loc(i)是第i个元素的存储地址，Loc(0)是数组的首地址，L是每个数据元素占用的字节数。

对于**二维数组**，采用行优先顺序进行存储，即先存储数组的第一行，再依次存储其他各行。对于一个n×m的数组A[n][m]，数组元素的存储地址为Loc(i,j)=Loc(0,0)+(i×m+j)×L，其中Loc(i,j)是第i行第j列的数组元素的存储地址，Loc(0,0)是数组的首地址，L是每个数据元素占用的字节数。

——4.3.2 数组的特性——

将计算数组元素的存储位置的公式推广到一般情况，可得n维数组A[m1][m2]...[mn]的数据元素的存储位置：

在n维数组中，计算数组中数据元素的存储地址的时间复杂度为O(1)，n维数组是一种随机存储结构。

——4.3.3 数组的遍历——

对二维数组进行遍历操作有两种次序，即行主序和列主序。

（1） 行主序： 以行序为主要次序，按行序递增访问数组的每行，同一行按列序递增访问数组元素。

（2） 列主序： 以列序为主要次序，按列序递增访问数组的每列，同一列按行序递增访问数组元素。

——4.3.3 数组的遍历——

举例：

请你设计一个算法，求二维数组A[n,n]的两条对角线元素之和。

答案示例：

```
1   def sumOfDiagonal(a):
2       n = len(a[0])
3       sum1 = sum2 = 0
4       for i in range(n):
5           sum1 += a[i][i]
6           sum2 += a[i][n-i-1]
7       sum = sum1 + sum2
8       if n%2 ==1:
9           sum -= a[n//2][n//2]
10      return sum
```

4.4 特殊矩阵的压缩存储

——4.4 特殊矩阵的压缩存储——

在科学技术和工程计算的许多领域，矩阵是数值分析问题研究的对象。特殊矩阵是具有许多相同数据元素或者零元素且数据元素的分布具有一定规律的矩阵，例如对称矩阵、三角矩阵和对角矩阵。

数据压缩技术是计算机软件领域研究的一个重要问题，图像、音频、视频等多媒体信息都需要进行数据压缩存储。本节将以特殊矩阵为例介绍矩阵的压缩存储。

矩阵采用二维数组进行存储，至少占用m×n个存储单元。当矩阵的阶数很大时，矩阵所占用的存储空间巨大，因此需要研究矩阵的压缩存储问题，根据不同矩阵的特点设计不同的压缩存储方法，节省存储空间，同时保证采用压缩存储的矩阵仍然能够正确地进行各种矩阵运算。

——4.4 特殊矩阵的压缩存储——

常用的矩阵压缩存储方法主要有以下两种：

(1) 对于零元素分布有规律的特殊矩阵，采用线性压缩或三角形的二维数组，只存储有规律的部分元素。

(2) 对于零元素分布没有规律的特殊矩阵，只存储非零元素。

下面，我们分别介绍一下不同类型的矩阵压缩存储方式

——4.4.1 三角矩阵的压缩存储——

三角矩阵包括上三角矩阵和下三角矩阵。假如是一个n阶矩阵，由n（n+1）/2个元素组成。当i<j时矩阵中的数据元素满足=0，矩阵为下三角矩阵； 当i>j时，矩阵中的数据元素满足=0，矩阵为上三角矩阵。

三角矩阵中具有近一半的分布有规律的零元素，所以三角矩阵采取只存储主对角线以及上/下三角部分的矩阵元素的压缩方法，主要分为以下两种：

1. 线性压缩存储

2. 使用三角形的二维数组压缩存储

——4.4.1 三角矩阵的压缩存储——

1．线性压缩存储

将下三角矩阵的主对角线及其以下元素按行主序顺序压缩成线性存储结构，存储元素的个数为n（n+1）/2，其中元素的存储地址如下：

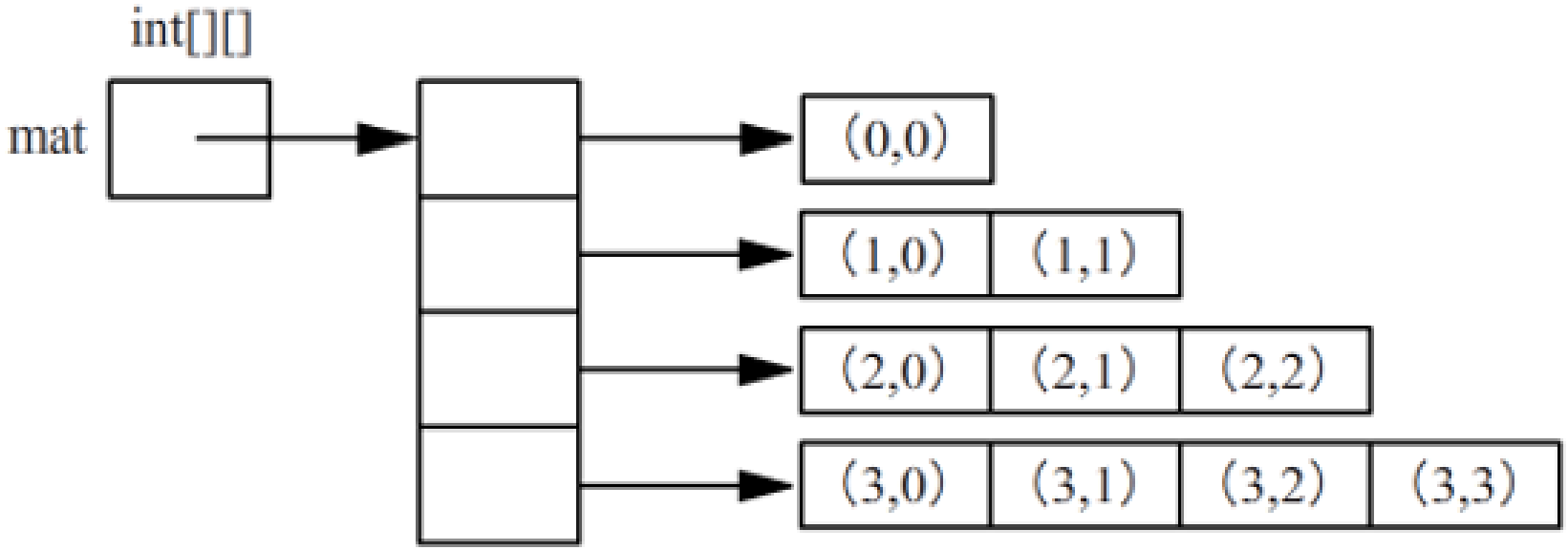
其中，注意L为数据元素所占据存储空间的字节数。

计算各数据元素的存储地址的时间复杂度为O（1）,三角矩阵的线性压缩存储结构是随机存储结构。

——4.4.1 三角矩阵的压缩存储——

2.使用三角形的二维数组压缩存储

三角形的二维数组实际上是一种动态数组结构，第i行一维数组的长度为i+1，存储在mat[i][j]中，如图4.4所示。计算各数据元素的存储地址的时间复杂度为O（1），此压缩存储结构是随机存储结构。



——4.4.2 对称矩阵的压缩存储——

n阶对称矩阵是指一个n阶矩阵中的数据元素满足ai,j=aj,i。对称矩阵在进行压缩存储时只存储主对角线和上/下部分数据元素，即将对称矩阵的主对角线及其上/下部分数据元素按行主序顺序压缩成线性存储，占用n（n+1）/2个存储单元,矩阵元素的线性压缩存储地址为：

——4.4.3 对角矩阵的压缩存储——

如果一个矩阵的所有非零元素都集中在以主对角线为中心的带状区域，则称该矩阵为对角矩阵。它是一个n阶矩阵，除了主对角线上的元素，其科元素均为0，则是主对角矩阵； 除了主对角线上及主对角线上下各一个元素外，其余元素均为0，为三对角矩阵。

在压缩存储对角矩阵时，只存储主对角线及其两侧部分的元素。如压缩存储主对角矩阵，将主对角元素顺序压缩成线性存储，存储元素个数为n，矩阵数据元素的线性压缩存储地址为：

——4.4.4 稀疏矩阵的压缩存储——

稀疏矩阵是指矩阵中的非零元素个数远远小于矩阵元素个数并且非零元素的分布没有规律的矩阵。设矩阵中有t个非零元素，非零元素占元素总数的比例称为矩阵的稀疏因子，通常稀疏因子小于0.05的矩阵称为稀疏矩阵。一般使用以下几种方法进行稀疏矩阵的压缩存储。

1. 稀疏矩阵的非零元素三元组
2. 稀疏矩阵的十字链表存储

——4.4.4 稀疏矩阵的压缩存储——

1．稀疏矩阵的非零元素三元组

稀疏矩阵的压缩存储原则是只存储矩阵中的非零元素，而仅存储非零元素是不够的，必须存储该元素在矩阵中的位置。矩阵元素的行号、列号和元素值称为该元素的三元组。

在Python语言中稀疏矩阵的三元组表示的结点结构定义如下：

```
1    class TripleNode(object):
2        def __init__(self,row=0,column=0,value=0):
3            self.row = row
4            self.column = column
5            self.value = value
```

——4.4.4 稀疏矩阵的压缩存储——

1．稀疏矩阵的非零元素三元组

稀疏矩阵的三元组顺序表类的定义如下：

```
class SparseMatrix(object):
    def __init__(self,maxSize):
        self.maxSize = maxSize
        self.data = [None] * self.maxSize # 三元组表
        for i in range(self.maxSize):
            self.data[i] = TripleNode()
        self.rows = 0 # 行数
        self.cols = 0 # 列数
        self.nums = 0 # 非零元素个数
```

——4.4.4 稀疏矩阵的压缩存储——

1．稀疏矩阵的非零元素三元组

初始化三元组顺序表是按先行序后列序的原则扫描稀疏矩阵，并把非零元素插入到顺序表中，其算法如下。

```
1   def create(self,mat):
2       count = 0
3       self.rows = len(mat)
4       self.cols = len(mat[0])
5       for i in range(self.rows):
6           for j in range(self.cols):
7               if mat[i][j]!=0:
8                   count += 1
9       self.num = count
10      self.data = [None] * self.nums
11      for i in range(self.rows):
12          for j in range(self.cols):
13              if mat[i][j]!=0:
14                  self.data[k] = TripleNode(i,j,mat[i][j])
15                  k += 1
```

——4.4.4 稀疏矩阵的压缩存储——

2. 稀疏矩阵的十字链表存储

当稀疏矩阵中非零元素的位置或个数经常发生变化时不宜采用三元组顺序表存储结构，而应该采用链式存储结构表示。十字链表是稀疏矩阵的另一种存储结构，在十字链表中稀疏矩阵的非零元素用一个结点来表示，每个结点由5个域组成。row域存放该元素的行号，column域存放该元素的列号，value域存放该元素的值，right域存放与该元素同行的下一个非零元素结点的指针，down域存放与该元素同列的下一个非零元素结点的指针。每个非零数据元素结点既是某个行链表中的一个结点，也是某个列链表中的结点，整个稀疏矩阵构成了一个十字交叉的链表，这样的链表就称为十字链表。

——4.4.4 稀疏矩阵的压缩存储——

2. 稀疏矩阵的十字链表存储

在Python语言中可以将稀疏矩阵的十字链表表示的结点结构定义如下：

```
class OLNode(object):
    def __init__(self,row=0,col=0,value=0):
        self.row = row # 行号
        self.col = col # 列号
        self.value = value # 数据元素值
        self.right = None # 行链表指针
        self.down = None # 列链表指针
```

稀疏矩阵的十字链表类的定义如右：

```
class CrossList(object):
    def __init__(self,rows,cols):
        self.rows = rows # 十字链表的行数
        self.cols = cols # 十字链表列数
        self.nums = 0 # 非零元素的个数
         # 行列的指针数组
        self.rhead = [None] * rows
        self.chead = [None] * cols
```

——4.4.4 稀疏矩阵的压缩存储——

下面，我们做一个例题，来深入了解不同存储结构的优缺点:

已知A为稀疏矩阵，试从空间和时间角度比较采用二维数组和三元组顺序表两种不同的存储结构完成求运算的优缺点。

参考答案:

设稀疏矩阵为m行n列，如果采用二维数组存储，其空间复杂度为O(m×n)； 因为要将所有的矩阵元素累加起来，所以需要用一个两层的嵌套循环，其时间复杂度也为O(m×n)。

如果采用三元组顺序表进行压缩存储，假设矩阵中有t个非零元素，其空间复杂度为O(t)，将所有的矩阵元素累加起来只需将三元组顺序表扫描一遍，其时间复杂度也为O(t)。当t<<m×n时采用三元组顺序表存储可获得较好的时空性能。

4.5 总结

——4.5 总结——

- (1) 字符串是数据元素类型为字符的线性表，串具有插入、删除、链接、查找、比较等基本操作。

  (2) 字符串具有顺序存储结构和链式存储结构两种存储结构。字符串的顺序存储结构叫顺序串，与顺序表的逻辑结构相同，存储结构类似，均可用数组来存储数据元素。字符串的链式存储结构叫链串，和线性表的链式存储结构类似，可以采用单链表存储串值。链串由一系列大小相同的结点组成，每个结点用数据域存放字符，指针域存放指向下一个结点的指针。

  (3) 串的模式匹配也叫查找定位，指的是在当前串中寻找模式串的过程，主要的模式匹配算法有Brute Force算法和KMP算法。

——4.5 总结——

- (4) 数组是n个具有相同数据类型的数据元素构成的集合，数组元素按某种次序存储在地址连续的存储单元中，是一种随机存储结构。

  (5) 特殊矩阵是具有许多相同数据元素或者零元素且数据元素的分布具有一定规律的矩阵，例如对称矩阵、三角矩阵和对角矩阵。为了节省存储空间，对矩阵进行压缩存储。特殊矩阵的压缩存储方法是将呈现规律性分布的、值相同的多个矩阵元素压缩存储到一个存储空间。

  (6) 稀疏矩阵是具有较多零元素，并且非零元素的分布无规律的矩阵。稀疏矩阵的压缩存储是只给非零数据元素分配存储空间。

谢谢观看